```
"""Customer Segmentation using RFM + KMeans (scikit-learn)"""
import pandas as pd, numpy as np
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt, seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA

# 1. Load data
df = pd.read_csv("Sales_Data.csv", parse_dates=["Order Date"])
snapshot = df["Order Date"].max() + pd.Timedelta(days=1)

# 2. Customer-level features (behaviour + demographics)
cust = df.groupby("Customer ID").agg(
    Recency=("Order Date", lambda x: (snapshot - x.max()).days),
    Frequency=("Order ID", "nunique"),
    Monetary=("Sales", "sum"),
    Avg_Discount=("Discount", "mean"),
    Total_Profit=("Profit", "sum"),
    Segment=("Segment", lambda x: x.mode()[0]),
    Region=("Region", lambda x: x.mode()[0]),
    Top_Category=("Category", lambda x: x.mode()[0]),
).reset_index()
cust["Avg_Order_Value"] = cust["Monetary"] / cust["Frequency"]

# 3. Scale + choose k
feats = ["Recency", "Frequency", "Monetary", "Avg_Discount"]
X = StandardScaler().fit_transform(np.log1p(cust[feats].clip(lower=0)))
ks = range(2, 9)
inertia, sil = [], []
for k in ks:
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    inertia.append(km.inertia_); sil.append(silhouette_score(X, km.labels_))
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].plot(ks, inertia, "o-"); ax[0].set(title="Elbow Method", xlabel="k", ylabel
ax[1].plot(ks, sil, "o-", color="green"); ax[1].set(title="Silhouette Score", xla
plt.tight_layout(); plt.savefig("1_elbow_silhouette.png", dpi=150); plt.close()

K = 4
km = KMeans(n_clusters=K, n_init=10, random_state=42).fit(X)
cust["Cluster"] = km.labels_
print("Silhouette (k=4):", round(silhouette_score(X, km.labels_), 3))
```

```
# 4. Name segments from cluster profiles
prof = cust.groupby("Cluster")[["Recency", "Frequency", "Monetary"]].mean()
names = {}
names[prof["Monetary"].idxmax()] = "Champions (High Value)"
rest = prof.drop(index=list(names))
names[rest["Recency"].idxmax()] = "At-Risk / Lost"
rest = prof.drop(index=list(names))
names[rest["Monetary"].idxmax()] = "Loyal Customers"
for c in prof.index:
    names.setdefault(c, "Occasional / New")
cust["Segment_Name"] = cust["Cluster"].map(names)

summary = cust.groupby("Segment_Name").agg(
    Customers=("Customer ID", "count"), Avg_Recency=("Recency", "mean"),
    Avg_Frequency=("Frequency", "mean"), Avg_Spend=("Monetary", "mean"),
    Avg_Discount=("Avg_Discount", "mean"), Total_Revenue=("Monetary", "sum"),
    Total_Profit=("Total_Profit", "sum")).round(2).sort_values("Total_Revenue", a
print(summary.to_string())
summary.to_csv("segment_summary.csv"); cust.to_csv("customers_segmented.csv", ind

# 5. Visuals
pal = sns.color_palette("Set2", K)
order = summary.index.tolist()
pca = PCA(2).fit_transform(X)
plt.figure(figsize=(7, 5))
sns.scatterplot(x=pca[:, 0], y=pca[:, 1], hue=cust["Segment_Name"], palette="Set2
plt.title("Customer Segments (PCA view)"); plt.xlabel("PC1"); plt.ylabel("PC2")
plt.tight_layout(); plt.savefig("2_segments_pca.png", dpi=150); plt.close()

fig, ax = plt.subplots(1, 3, figsize=(15, 4))
for a, (col, t) in zip(ax, [("Recency", "Avg Recency (days)"), ("Frequency", "Avg
    sns.barplot(data=cust, x="Segment_Name", y=col, order=order, palette="Set2",
    a.set_title(t); a.set_xlabel(""); a.tick_params(axis="x", rotation=25)
plt.tight_layout(); plt.savefig("3_rfm_by_segment.png", dpi=150); plt.close()

fig, ax = plt.subplots(1, 2, figsize=(12, 4))
cust["Segment_Name"].value_counts().plot.pie(autopct="%1.0f%%", ax=ax[0], colors=
ax[0].set_title("Customer Share by Segment")
summary["Total_Revenue"].plot.bar(ax=ax[1], color=pal); ax[1].set_title("Revenue
ax[1].tick_params(axis="x", rotation=25)
plt.tight_layout(); plt.savefig("4_size_revenue.png", dpi=150); plt.close()

fig, ax = plt.subplots(1, 2, figsize=(12, 4))
pd.crosstab(cust["Segment_Name"], cust["Region"], normalize="index").plot.bar(sta
ax[0].set_title("Region Mix"); ax[0].set_xlabel("")
pd.crosstab(cust["Segment_Name"], cust["Top_Category"], normalize="index").plot.b
ax[1].set_title("Preferred Category"); ax[1].set_xlabel("")
```

```
for a in ax: a.tick_params(axis="x", rotation=25)
plt.tight_layout(); plt.savefig("5_demographics_preferences.png", dpi=150); plt.c
print("Done")
```